## 16-AprEndpoint组件：TomcatAPR提高IO性能的秘密

我们在使用Tomcat时，会在启动日志里看到这样的提示信息：

> The APR based Apache Tomcat Native library which allows optimal performance in production environments was not found on the java.library.path: ***

这句话的意思就是推荐你去安装APR库，可以提高系统性能。那什么是APR呢？

APR（Apache Portable Runtime Libraries）是Apache可移植运行时库，它是用C语言实现的，其目的是向上层应用程序提供一个跨平台的操作系统接口库。Tomcat可以用它来处理包括文件和网络I/O，从而提升性能。我在专栏前面提到过，Tomcat支持的连接器有NIO、NIO.2和APR。跟NioEndpoint一样，AprEndpoint也实现了非阻塞I/O，它们的区别是：NioEndpoint通过调用Java的NIO API来实现非阻塞I/O，而AprEndpoint是通过JNI调用APR本地库而实现非阻塞I/O的。

那同样是非阻塞I/O，为什么Tomcat会提示使用APR本地库的性能会更好呢？这是因为在某些场景下，比如需要频繁与操作系统进行交互，Socket网络通信就是这样一个场景，特别是如果你的Web应用使用了TLS来加密传输，我们知道TLS协议在握手过程中有多次网络交互，在这种情况下Java跟C语言程序相比还是有一定的差距，而这正是APR的强项。

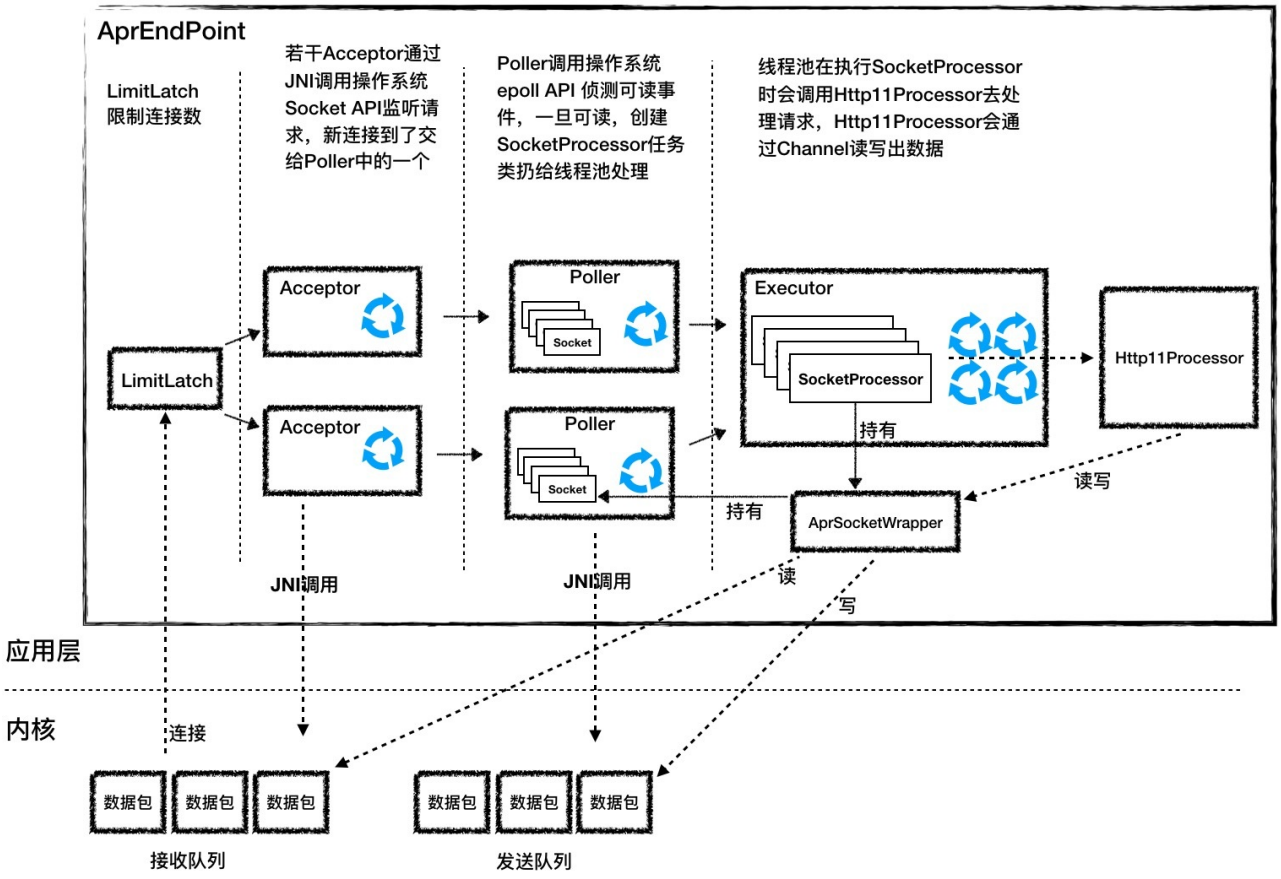
Tomcat本身是Java编写的，为了调用C语言编写的APR，需要通过JNI方式来调用。JNI（Java Native Interface）是JDK提供的一个编程接口，它允许Java程序调用其他语言编写的程序或者代码库，其实JDK本身的实现也大量用到JNI技术来调用本地C程序库。

在今天这一期文章，首先我会讲AprEndpoint组件的工作过程，接着我会在原理的基础上分析APR提升性能的一些秘密。在今天的学习过程中会涉及到一些操作系统的底层原理，毫无疑问掌握这些底层知识对于提高你的内功非常有帮助。

### AprEndpoint工作过程

下面我还是通过一张图来帮你理解AprEndpoint的工作过程。

仅供个人学习 请勿传播 用户标记 mannix小查理




你会发现它跟NioEndpoint的图很像，从左到右有LimitLatch、Acceptor、Poller、SocketProcessor和Http11Processor，只是Acceptor和Poller的实现和NioEndpoint不同。接下来我分别来讲讲这两个组件。

### Acceptor

Accpetor的功能就是监听连接，接收并建立连接。它的本质就是调用了四个操作系统API：socket、bind、listen和accept。那Java语言如何直接调用C语言API呢？答案就是通过JNI。具体来说就是两步：先封装一个Java类，在里面定义一堆用**native关键字**修饰的方法，像下面这样。

```
public class Socket {
  ...
  //用native修饰这个方法，表明这个函数是C语言实现
  public static native long create(int family, int type,
                                   int protocol, long cont)

  public static native int bind(long sock, long sa);

  public static native int listen(long sock, int backlog);

  public static native long accept(long sock)
}
```

接着用C代码实现这些方法，比如bind函数就是这样实现的：

```
//注意函数的名字要符合JNI规范的要求
JNIEXPORT jint JNICALL
Java_org_apache_tomcat_jni_Socket_bind(JNIEnv *e, jlong sock,jlong sa)
```

```
{
    jint rv = APR_SUCCESS;
    tcn_socket_t *s = (tcn_socket_t *) sock;
    apr_sockaddr_t *a = (apr_sockaddr_t *) sa;

       //调用APR库自己实现的bind函数
    rv = (jint)apr_socket_bind(s->sock, a);
    return rv;
}
```

专栏里我就不展开JNI的细节了，你可以扩展阅读获得更多信息和例子。我们要注意的是函数名字要符合JNI的规范，以及Java和C语言如何互相传递参数，比如在C语言有指针，Java没有指针的概念，所以在Java中用long类型来表示指针。AprEndpoint的Acceptor组件就是调用了APR实现的四个API。

### Poller

Acceptor接收到一个新的Socket连接后，按照NioEndpoint的实现，它会把这个Socket交给Poller去查询I/O事件。AprEndpoint也是这样做的，不过AprEndpoint的Poller并不是调用Java NIO里的Selector来查询Socket的状态，而是通过JNI调用APR中的poll方法，而APR又是调用了操作系统的epoll API来实现的。

这里有个特别的地方是在AprEndpoint中，我们可以配置一个叫deferAccept的参数，它对应的是TCP协议中的TCP_DEFER_ACCEPT，设置这个参数后，当TCP客户端有新的连接请求到达时，TCP服务端先不建立连接，而是再等等，直到客户端有请求数据发过来时再建立连接。这样的好处是服务端不需要用Selector去反复查询请求数据是否就绪。

这是一种TCP协议层的优化，不是每个操作系统内核都支持，因为Java作为一种跨平台语言，需要屏蔽各种操作系统的差异，因此并没有把这个参数提供给用户；但是对于APR来说，它的目的就是尽可能提升性能，因此它向用户暴露了这个参数。

## APR提升性能的秘密

APR连接器之所以能提高Tomcat的性能，除了APR本身是C程序库之外，还有哪些提速的秘密呢？

### JVM堆 VS 本地内存

我们知道Java的类实例一般在JVM堆上分配，而Java是通过JNI调用C代码来实现Socket通信的，那么C代码在运行过程中需要的内存又是从哪里分配的呢？C代码能否直接操作Java堆？

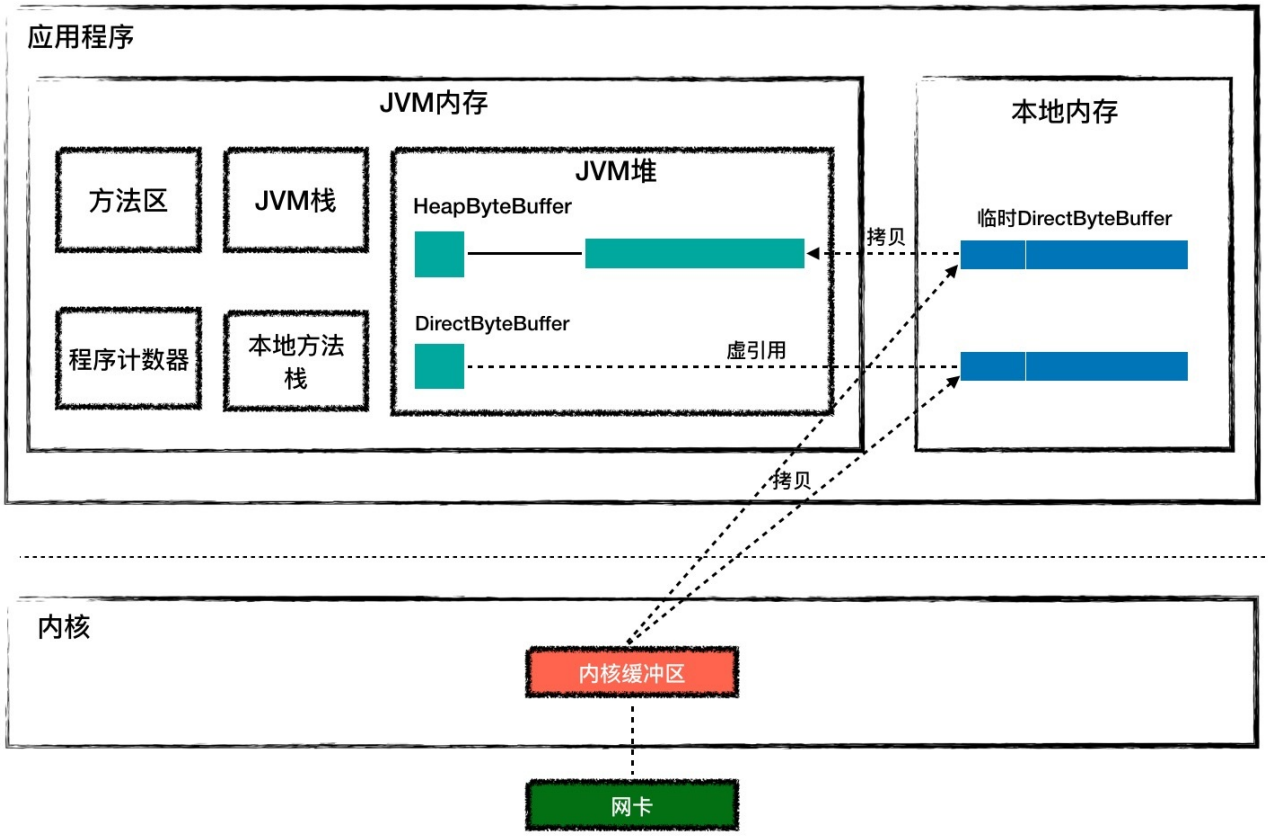
为了回答这些问题，我先来说说JVM和用户进程的关系。如果你想运行一个Java类文件，可以用下面的Java命令来执行。

```
java my.class
```

这个命令行中的java其实是**一个可执行程序，这个程序会创建JVM来加载和运行你的Java类**。操作系统会创建一个进程来执行这个java可执行程序，而每个进程都有自己的虚拟地址空间，JVM用到的内存（包括

仅供个人学习 请勿传播 用户标记 mannix小童鞋

堆、栈和方法区）就是从进程的虚拟地址空间上分配的。请你注意的是，JVM内存只是进程空间的一部分，除此之外进程空间内还有代码段、数据段、内存映射区、内核空间等。从JVM的角度看，JVM内存之外的部分叫作本地内存，C程序代码在运行过程中用到的内存就是本地内存中分配的。下面我们通过一张图来理解一下。




Tomcat的Endpoint组件在接收网络数据时需要预先分配好一块Buffer，所谓的Buffer就是字节数组byte[]，Java通过JNI调用把这块Buffer的地址传给C代码，C代码通过操作系统API读取Socket并把数据填充到这块Buffer。Java NIO API提供了两种Buffer来接收数据：HeapByteBuffer和DirectByteBuffer，下面的代码演示了如何创建两种Buffer。

```
//分配HeapByteBuffer
ByteBuffer buf = ByteBuffer.allocate(1024);

//分配DirectByteBuffer
ByteBuffer buf = ByteBuffer.allocateDirect(1024);
```

创建好Buffer后直接传给Channel的read或者write函数，最终这块Buffer会通过JNI调用传递给C程序。

```
//将buf作为read函数的参数
int bytesRead = socketChannel.read(buf);
```

那HeapByteBuffer和DirectByteBuffer有什么区别呢？HeapByteBuffer对象本身在JVM堆上分配，并且它持有的字节数组byte[]也是在JVM堆上分配。但是如果用**HeapByteBuffer**来接收网络数据，**需要把数据从内核先拷贝到一个临时的本地内存，再从临时本地内存拷贝到JVM堆**，而不是直接从内核拷贝到JVM堆上。这

是为什么呢？这是因为数据从内核拷贝到JVM堆的过程中，JVM可能会发生GC，GC过程中对象可能会被移动，也就是说JVM堆上的字节数组可能会被移动，这样的话Buffer地址就失效了。如果这中间经过本地内存中转，从本地内存到JVM堆的拷贝过程中JVM可以保证不做GC。

如果使用HeapByteBuffer，你会发现JVM堆和内核之间多了一层中转，而DirectByteBuffer用来解决这个问题，DirectByteBuffer对象本身在JVM堆上，但是它持有的字节数组不是从JVM堆上分配的，而是从本地内存分配的。DirectByteBuffer对象中有个long类型字段address，记录着本地内存的地址，这样在接收数据的时候，直接把这个本地内存地址传递给C程序，C程序会将网络数据从内核拷贝到这个本地内存，JVM可以直接读取这个本地内存，这种方式比HeapByteBuffer少了一次拷贝，因此一般来说它的速度会比HeapByteBuffer快好几倍。你可以通过上面的图加深理解。

Tomcat中的AprEndpoint就是通过DirectByteBuffer来接收数据的，而NioEndpoint和Nio2Endpoint是通过HeapByteBuffer来接收数据的。你可能会问，NioEndpoint和Nio2Endpoint为什么不用DirectByteBuffer呢？这是因为本地内存不好管理，发生内存泄漏难以定位，从稳定性考虑，NioEndpoint和Nio2Endpoint没有去冒这个险。

**sendfile**

我们再来考虑另一个网络通信的场景，也就是静态文件的处理。浏览器通过Tomcat来获取一个HTML文件，而Tomcat的处理逻辑无非是两步：

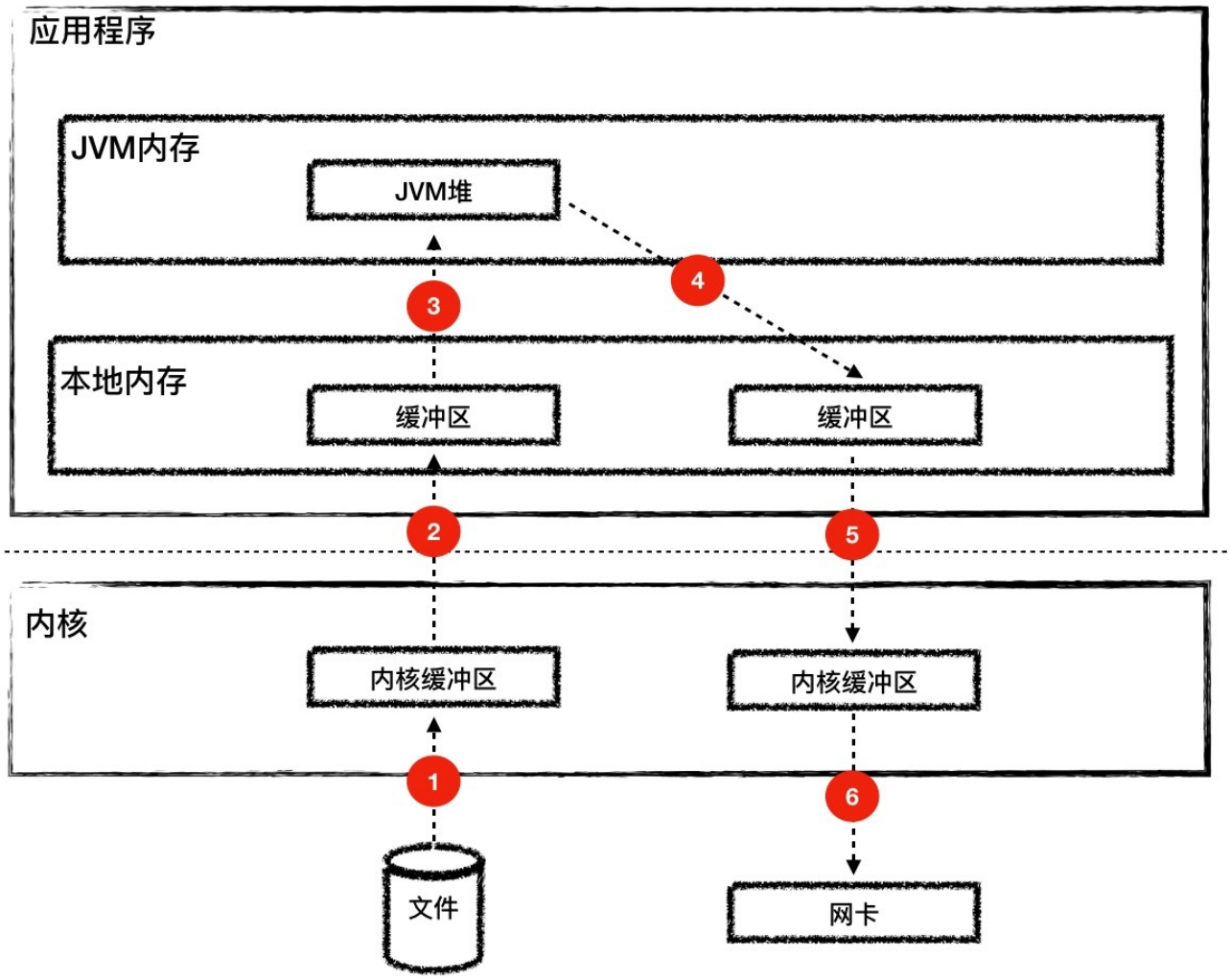
1. 从磁盘读取HTML到内存。
2. 将这段内存的内容通过Socket发送出去。

但是在传统方式下，有很多次的内存拷贝：

- 读取文件时，首先是内核把文件内容读取到内核缓冲区。
- 如果使用HeapByteBuffer，文件数据从内核到JVM堆内存需要经过本地内存中转。
- 同样在将文件内容推入网络时，从JVM堆到内核缓冲区需要经过本地内存中转。
- 最后还需要把文件从内核缓冲区拷贝到网卡缓冲区。

从下面的图你会发现这个过程有6次内存拷贝，并且read和write等系统调用将导致进程从用户态到内核态的切换，会耗费大量的CPU和内存资源。

仅供个人学习 请勿传播 用户标记 mannix小查理



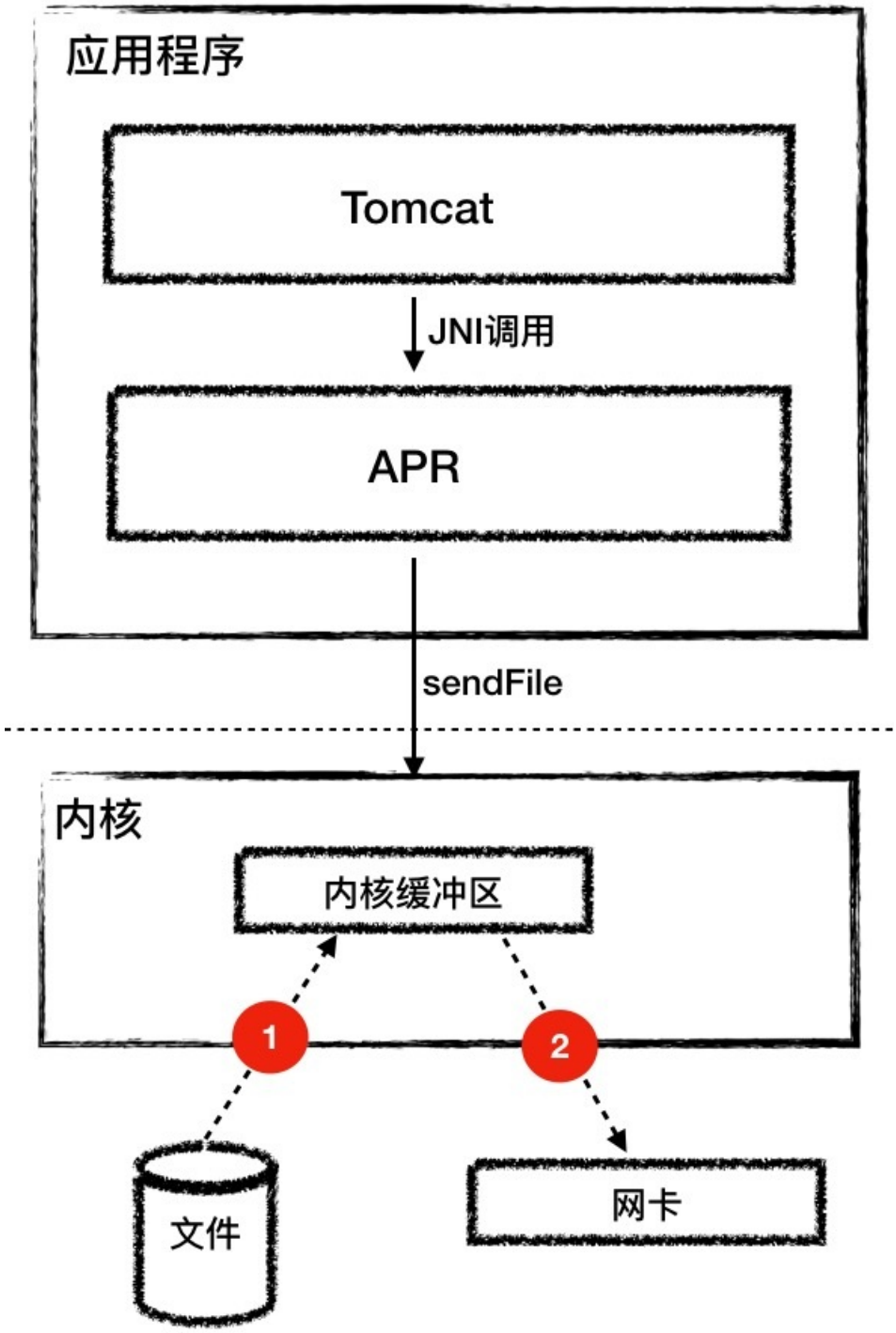

而Tomcat的AprEndpoint通过操作系统层面的sendfile特性解决了这个问题，sendfile系统调用方式非常简洁。

```
sendfile(socket, file, len);
```

它带有两个关键参数：Socket和文件句柄。将文件从磁盘写入Socket的过程只有两步：

第一步：将文件内容读取到内核缓冲区。

第二步：数据并没有从内核缓冲区复制到Socket关联的缓冲区，只有记录数据位置和长度的描述符被添加到Socket缓冲区中；接着把数据直接从内核缓冲区传递给网卡。这个过程你可以看下面的图。




### 本期精华

对于一些需要频繁与操作系统进行交互的场景，比如网络通信，Java的效率没有C语言高，特别是TLS协议握手过程中需要多次网络交互，这种情况下使用APR本地库能够显著提升性能。

除此之外，APR提升性能的秘密还有：通过DirectByteBuffer避免了JVM堆与本地内存之间的内存拷贝；通过sendfile特性避免了内核与应用之间的内存拷贝以及用户态和内核态的切换。其实很多高性能网络通信组件，比如Netty，都是通过DirectByteBuffer来收发网络数据的。由于本地内存难于管理，Netty采用了本地内存池技术，感兴趣的同学可以深入了解一下。

### 课后思考

为什么不同的操作系统，比如Linux和Windows，都有自己的Java虚拟机？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得

仅供个人学习 请勿传播 用户标记 mannix小查理

记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。




**精选留言：**

- YF 2019-06-15 11:27:57
  老师，为什么从本地内存到 JVM 堆的拷贝过程中 JVM 可以保证不做 GC呢？那为什么从内核拷贝到JVM堆中就不能保证做GC呢？ [2赞]

  作者回复2019-06-15 21:30:54
  这是HotSpot VM层面保证的，具体来说就是HotSpot不是什么时候都能GC的，需要JVM中各个线程都处在“safepoint”时才能GC， 本地内存到JVM 堆的拷贝过程中是没有“safepoint”的，所以不会GC，至于什么是“safepoint”，你可以搜索一下。

- nimil 2019-06-15 09:03:58
  学到了，之前还纳闷，我在使用高版本的SpringBoot的时候总是会有arp的警告，今天终于知道是干嘛的了。感谢老师。 [2赞]

- 刘章周 2019-06-15 08:50:45
  java语言有个特性，一次编译到处执行，而class文件需要通过虚拟机编译成操作系统指令。不同操作系统的指令不一样，所以有对应的虚拟机来简化代码开发。c语言好像是直接调用操作系统指令，代码开发中调用一个方法，不同的操作系统可能不一样，还得准备两份。老师，说的对吗？ [1赞]

  作者回复2019-06-15 11:38:53
  对的

- nightmare 2019-06-15 08:02:26
  老师，arp模式更加适合https，或者文件传输的业务场景，相对于nio会有更快的io速度 [1赞]

- 粉条炒肉 2019-06-15 07:09:30
  老师，请教一个问题，每次服务的访问高峰期重启或者发布tomcat服务，都会出现阻塞线程过多 这种情况是不是容器初始化的时候需要预热很多资源，从而响应不过来请求而导致阻塞，关于这种情况，有什么好的解决办法。 [1赞]

作者回复2019-06-15 21:50:50
服务没有启动完成的时候，不要把流量送过来。

- 飞翔 2019-06-16 13:02:06
老师 能不能讲讲什么时候用apr 什么时候用nio2 nio？

作者回复2019-06-16 17:21:31
如果web应用使用tls通信，对性能响应时间要求比较高，而用户又愿意花时间去配置APR，推荐APR。

NIO和NIO2其实差别不是很大，如果硬要区分，NIO适合处理比较轻的，数据传输量比较少的请求，AIO适合比较重，数据传输量比较大的请求。

这是因为NIO本质还是同步，数据从用户空间和内核空间之间的拷贝还是阻塞的。

- Liam 2019-06-16 09:58:51
hi，老师你好，能否讲讲零拷贝和文件内存映射机制的原理，和今天讲的buffer少一次copy是否有关？

作者回复2019-06-17 21:08:24
已经回复类似问题，在答疑篇详细分析

- z.l 2019-06-15 22:57:02
请教老师, DirectByteBuffer和常说的mmap是什么关系？

作者回复2019-06-17 21:07:39
没有关系的，mmap是将文件映射到用户进程的虚拟地址空间，进程向这个地址读写内存，内核负责将数据同步到磁盘文件，考虑在答疑篇详细说说

- 陆离 2019-06-15 09:50:45
虚拟机只是抽象了操作系统，但并不是虚拟机也是一套的。
jvm的解释器在不同操作系统下是不同的，因为要将字节码解释生成本地机器码。
这么简单个问题，感觉还是解释不清楚，这个要和操作系统原理联系起来了。
烦请老师做一个详细的简单！

作者回复2019-06-15 21:42:38
JVM为了跟操作系统打交道，必须调用操作系统提供的API，而每个操作系统提供的API都不同，所以必须针对不同的操作系统实现不同的JVM，这样位于JVM之上的Java字节码才可以实现跨平台。